

第四届环球杯



第12赛段：上海大奖赛

January 10-11, 2026

该习题集应包含 21 页编号的 13 个问题。

Based on



International Collegiate Programming Contest (ICPC)



问题A. 门吉，我们想念你！

时间限制：1 秒内存限制：1024 MB

这是一个交互问题。

门吉迷路了，每个人都想念他。找到他是你的工作！

有一个二叉树 T ，由 n 个顶点组成，树 T 的每条边的长度为 1。Menji 隐藏在顶点 X 处。你知道树的结构。然而，你不知道 X 。

要找到 X ，您可以发送信号。您可以选择一个顶点 u 并选择信号强度 k ，然后从顶点 u 发送强度为 k 的信号。如果 u 和 X 之间的距离不超过 k ，Menji 将收到信号并发送回信号，您将收到信号。否则，您将不会收到任何东西。

发送信号较慢，而您又很赶时间，所以请在不超过 40 个信号中确定门吉的位置。，所以请在不超过 40 个信号中确定门吉的位置。

交互协议

输入包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 100$)，即测试用例的数量。

对于每个测试用例，第一行包含一个整数 n ($2 \leq n \leq 3 \times 10^4$)，即树中的顶点数 e。

第二行包含 $n - 1$ 个整数 $fa_2, fa_3, \dots, fa_n$ ($1 \leq fa_i < i$)，其中 fa_i 是 i 的父代 n
树。该树的根位于顶点 1。

保证树是二叉树，即不存在 $1 < i < j < k \leq$ n , , ,
这样 $fa_i = fa_j = fa_k$ 。

要发送信号，请按以下格式打印一行：

- ? u k : 表示您在顶点 u 处创建了强度为 k 的信号。您需要确保 $1 \leq u \leq n, 0 \leq k \leq n$ 。然后你必须读取一个整数 o ($o \in \{0, 1\}$)。如果您收到信号，或者等效地， $dis(u, X) \leq k$ ，则 $o = 1$ ，否则 $o = 0$ 。

要报告答案，请按以下格式打印一行：

- ! u : 表明您找到了 $X = u$ 。打印此测试用例后，您需要继续进行下一个测试用例，或者如果没有更多测试用例则终止。

对于每个测试用例，您最多可以发送 40 个信号。报告答案不算发送信号。

如果您发送的信号超过 40 个，或者您发送的信号格式错误，或者您报告的答案不正确，则交互将结束，您将收到错误答案判决。

请注意，交互器是自适应的，这意味着答案可能会根据您的查询而变化，只要它与先前查询的约束和答案保持一致即可。

保证所有测试用例的 n 之和不超过 3×10^4 。

打印每一行后，不要忘记输出行尾并刷新输出。您可以使用 `fflush(stdout)` 或 `cout.flush()` 来刷新 C++ 的流，对于 Java 使用 `System.out.flush()`，对于 Python 使用 `stdout.flush()`。



例子

standard input	standard output
2	? 2 1
7	
1 1 2 2 3 3	? 3 2
1	? 4 0
0	! 5
0	? 2 1
7	? 3 0
1 1 2 2 3 3	! 3
0	
1	



问题 B.Hamu

时间限制：1 秒内存限制：1024 MB

叮咚打算去哈姆国旅行。

Hamu 国家由 n 个城市和 m 个双向道路组成。在这次旅行之前，叮咚已经访问了城市 i 整整 a_i 次。此次行程，叮咚计划进入 s 城市哈木。接下来的每一天，叮咚都会沿着某条路行驶到一个城市，并访问该城市一次。最后一天的访问结束时，Dingdong 应该到达城市 s ，并从城市 s 离开 Hamu。注意，进入哈姆国当天，叮咚并没有访问城市 s 。

叮咚希望，这次旅行之后，结合之前的访问，哈姆的每个城市他都去过偶数次。叮咚时间有限，此行最多可游览 $5n$ 个城市。请帮助他制定有效的旅行计划，如果不可能，请告诉他。

输入

输入包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 2 \times 10^5$)，测试用例的数量。

对于每个测试用例，第一行包含三个整数 n, m, s ($1 \leq n \leq 2 \times 10^5, 0 \leq m \leq 2 \times 10^5, 1 \leq s \leq n$)，其中 n 是城市数量， m 是道路数量， s 是起始城市的索引。

下一行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$)，叮咚访问的次数对于每个城市。

接下来的 m 行每行包含两个整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示连接城市 u_i 和城市 v_i 的无向道路。可以有多个边和自环。换句话说，不能保证 $u_i \neq v_i$ ，也不能保证 $i \neq j$ 是 $(u_i, v_i) \neq (u_j, v_j)$ 。

保证所有测试用例的 n 之和和 m 之和分别不超过 2×10^5 。

输出

对于每个测试用例，如果没有有效的旅行计划，请在一行中打印“否”。

否则，先在一行中打印 Yes，然后在下一行中打印一个整数 k ($0 \leq k \leq 5n$)，代表此行程的总访问次数。在下面的行中，按顺序打印访问过的城市的索引。



例子

standard input	standard output
5	Yes
4 4 1	4
1 1 1 1	2 3 4 1
1 2	Yes
2 3	6
3 4	2 4 5 2 3 1
4 1	Yes
5 7 1	0
9 4 3 11 7	
1 2	No
2 5	Yes
2 4	2
3 4	2 1
2 3	
1 3	
4 5	
2 2 2	
114 514	
1 2	
1 2	
5 0 1	
114 514 19 19 810	
2 1 1	
3 5	
1 2	

笔记

对于第一个测试用例，沿着路线 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$ 访问后，访问了城市 1, 2, 3, 4, 2, 2, 2, 各2次，满足要求。

d



问题 C. 奇点

时间限制：1 秒内存限制：1024 MB

2077 年，解决问题变得简单。机器人会通过设置一些随机操作产生一个问题，然后解决它。问题提出者只需检查问题是否正确。

这是2077年的一个问题：

给定排列 $p_1, p_2, \dots, p_n$ ，保证 n 是偶数。您希望对排列进行排序
仅使用一种类型的操作：

- FakeSort(l, r): 你必须保证 $r - l + 1$ 是偶数。令 $k = r - l + 1$ 为1，则连续子序列 $p_l, p_{l+1}, \dots, p_r$ 中最大的 $k/2$ 个元素和最小的 $k/2$ 个元素将独立排序。也就是说，令 $L_1, L_2, \dots, L_{k/2}$ 为最大 $k/2$ 个数的索引，而 $S_1, S_2, \dots, S_{k/2}$ 为最小 $k/2$ 个数的索引。我们首先对索引 $L_1 \sim L_{k/2}$ 上的数字进行排序，然后对索引 $S_1 \sim S_{k/2}$ 上的数字进行排序。

这是一个具体的例子；假设排列为 $p = \{2, 5, 7, 1, 8, 6, 4, 3\}$ 。如果我们调用 FakeSort(2,7) :

- $k = r - l + 1 = 6$ 、连续子序列 $p_l, p_{l+1}, \dots, p_r$ 为 $\{5, 7, 1, 8, 6, 4\}$ ；我们将对该序列中最大的 3 个数字和最小的 3 个数字独立排序。
- $p = \{2, 5, 7, 1, 8, 6, 4, 3\}$ ；最大的 3 个数字是粗体的。排序后变为 $p = \{2, 5, 6, 1, 7, 8, 4, 3\}$ 。
- $p = \{2, 5, 6, 1, 7, 8, 4, 3\}$ ；最小的 3 个数字是粗体的。排序后变为 $p = \{2, 1, 6, 4, 7, 8, 5, 3\}$ 。

所以 FakeSort(2,7) 之后的 $p = \{2, 5, 7, 1, 8, 6, 4, 3\}$ 变为 $\{2, 1, 6, 4, 7, 8, 5, 3\}$ 。

请使用不超过 114 个运算来对排列进行排序，否则确定不可能。可以证明，如果可以用这个操作对排列进行排序，就有办法使用不超过114个操作。

输入

输入包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 10^3$)，即测试用例的数量。

对于每个测试用例，第一行包含一个偶数 n ($4 \leq n \leq 10^5$)，即排列的长度 n 。

第二行包含 n 整数 $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq n$)，即您需要排序的排列。保证 p 是一个排列。

保证所有测试用例的 n 之和不超过 2×10^5 。

输出

对于每个测试用例，如果无法对排列进行排序，则打印 -1 。否则，打印一个整数 k ($0 \leq k \leq 114$)，表示使用的操作数。在接下来的 k 行中，每行打印 l, r ($1 \leq l \leq r \leq n, r - l + 1$ 是偶数)，表示执行 Fakesort(l, r)。



例子

standard input	standard output
3	1
4	1 4
2 1 4 3	-1
6	2
3 6 5 1 2 4	4 7
8	1 6
3 2 1 7 5 4 6 8	



问题 D. 不是子集和

时间限制：1 秒内存限制：1024 MB

我们得到一个长度为 2^n 的数组 $a_0, a_1, \dots, a_{2^n-1}$ 。对于字母表 $0,1,?$ 上长度为 n 的字符串 q , (1-索引), 将“广义子集和” $S(q)$ 定义为满足以下条件的所有索引 j 上的 a_j 之和:

- 对于每个 $1 \leq k \leq n$, 如果 $q_k = 0$, 则 j 的第 k 位为 0。
- 对于每个 $1 \leq k \leq n$, 如果 $q_k = 1$, 则 j 的第 k 位为 1。
- 如果 $q_k = ?$ 对 j 的第 k 位没有限制。

例如, 当 $n = 3$ 且 $s = 0 \leq 1$ 时, 广义子集和为 $S(q) = a_4 + a_6$ (二进制 100 和 110)。请注意, 第一位是最低位。

您的任务是计算字母表 $0,1,?$ 上每个长度为 n 的字符串的广义子集和。由于总输出可能很大, 因此您只需输出所有这些和的按位异或即可。

输入

输入的第一行包含一个整数 n ($1 \leq n \leq 16$)。

输入的第二行包含 2^n 个整数 $a_0, a_1, \dots, a_{2^n-1}$ ($0 \leq a_i \leq 9$), 内容在数组 a 中。

输出

打印一个整数, 表示所有广义子集和的按位异或。

例子

standard input	standard output
1 3 5	14
2 2 6 8 8	0

笔记

以下是示例 2 中的查询字符串 q 及其对应的广义子集和 $S(q)$:

$$q = 00, S(q) = a_0 = 2$$

$$q = 10, S(q) = a_1 = 6$$

$$q = ?0, S(q) = a_0 + a_1 = 2 + 6 = 8$$

$$q = 01, S(q) = a_2 = 8$$

$$q = 11, S(q) = a_3 = 8$$

$$q = ?1, S(q) = a_2 + a_3 = 16$$

$$q = 0?, S(q) = a_0 + a_2 = 10$$

$$q = 1?, S(q) = a_1 + a_3 = 14$$

$$q = ??, S(q) = a_0 + a_1 + a_2 + a_3 = 24$$

现在很容易验证输出是否为 0。



问题E. 花之国3

时间限制：1 秒内存限制：1024 MB

磁盘 #1 上存储有一个长度为 m 的二进制字符串 s_1 。
然后，有 $n - 1$ 操作。在第 i 次操作中，选择磁盘 $1 \leq p_{i+1} \leq i$ ，并将该磁盘上的字符串 $s_{p_{i+1}}$ 复制到磁盘 $i + 1$ ，生成 s_{i+1} 。然而，在复制过程中，最多可以翻转 k 位（即，最多在 k 位置出现错误）。
您将获得所有最终的 n 二进制字符串 $s_1, s_2, \dots, s_n$ ，每个字符串的长度为 m 。你的任务是确定许多可能的序列 $p_2, p_3, \dots, p_n$ 都可能导致最终的配置。
由于答案可能很大，因此您只需对 998244353 求模即可。

输入

输入的第一行包含三个整数 n, m, k ($2 \leq n \leq 5000, 4 \leq m \leq 15000, 1 \leq k \leq 3$)，如语句中所述。保证 m 是 4 的倍数。
接下来的每一行 n 都包含长度为 $m/4$ 的十六进制字符串 s'_i 。 s'_i 的每个字符都是 0-9 或 A-F 之一，其中 A = 10、B = 11、...、F = 15。
十六进制表示形式 s'_i 中的每一位对应于二进制串 s_i 中的四个连续位。具体地，对于每一位 $s'_{i,j}$ ，可以证明存在唯一满足 $s'_{i,j} = 8a + 4b + 2c + d$ 和 $a, b, c, d \in \{0, 1\}$ 的元组 (a, b, c, d) 。二进制串 s_i 中的位满足 $(s_{i,4j}, s_{i,4j+1}, s_{i,4j+2}, s_{i,4j+3}) = (a, b, c, d)$ 。

输出

打印一个整数 — 有效序列 $(p_2, p_3, \dots, p_n)$ 的数量以 998244353 为模。

例子

standard input	standard output
5 8 2 95 05 BD 9C BD	6



问题F. 花之国4

时间限制：1.5 秒内存限制：2048 M
B

2D 平面上有 n 段。每个线段从 x 轴的非负部分开始，到 y 轴的非负部分结束。换句话说，其起点的坐标为 $(x_i, 0)$, $x_i \geq 0$, 终点的坐标为 $(0, y_i)$, $y_i \geq 0$ 。

您将收到 q 查询。在每个查询中，指定一个线段，其起点位于 x 轴上，其终点可以是第一象限中的任何位置或平面轴的非负部分。对于每个查询线段，确定它是否与任何现有线段相交。计算端点处的交点。

查询之间是相互独立的；也就是说，每个查询中给出的段将不会保留在其余查询中。

输入

输入包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 10^6$)，表示测试用例的数量。

对于每个测试用例，第一行包含两个整数 n, q ($1 \leq n, q \leq 10^6$)，即现有段的数量以及查询的数量。

接下来的每一行 n 都包含两个整数 x_i, y_i ($0 \leq x_i, y_i \leq 10^9$)，描述了一个星段在 $(x_i, 0)$ 处并在 $(0, y_i)$ 处结束。

那么接下来的 q 行每一行都包含三个整数 a_j, b_j, c_j ($0 \leq a_j, b_j, c_j \leq 10^9$)，描述从 $(a_j, 0)$ 开始到 (b_j, c_j) 结束的查询段。

保证所有测试用例的 n 之和和 q 之和分别不超过 10^6 。

输出

对于每个查询，如果查询线段与至少一个现有线段相交（包括在端点处），则打印 YES，否则打印 NO。

例子

standard input	standard output
1	NO
3 5	YES
6 6	NO
2 6	YES
6 2	NO
10 4 4	
10 3 3	
0 1 1	
0 2 2	
5 2 1	



问题 G.Gemcrate

时间限制：1 秒内存限制：512 MB

黑色有 n 宝石。第 i 个宝石上写有一个正整数 a_i 。诺尔想要瓜分这些宝石
分成几个非空组。每颗宝石都属于一组。

s

假设第 i 组包含标签为 $k_{i,1}, k_{i,2}, \dots, k_{i,p}$ 的宝石，则 Noir 将第 i 组的亮度视为 $a_{k_{i,1}} \oplus a_{k_{i,2}} \oplus \dots \oplus a_{k_{i,p}}$ ，
其中 $\oplus$ 是按位异或运算。将第 i 组的亮度表示为 B_i 。

对于 m 组的分组方法，Noir 将该方法的值视为 $B_1 \& B_2 \& \dots \& B_m$ ，其中 $\&$ 是按位与运算。

Noir 希望找到所有分组方法中的最大可能值。

输入

输入包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 10^4$)，即测试用例的数量。

对于每个测试用例，第一行包含一个整数 n ($1 \leq n \leq 5 \times 10^5$)，即宝石数量。

第二行包含 n 整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i < 2^{60}$)，即写在宝石上的整数。

保证所有测试用例的 n 之和不超过 5×10^5 。

输出

对于每个测试用例，打印一个整数，表示所有分组方法的最大可能值

。

例子

standard input	standard output
4	2
4	6
1 2 3 1	26
6	13121614001578
4 7 5 2 6 3	
4	
14 15 9 18	
2	
251508091405 13011908091815	

笔记

对于第一个测试用例，可能的分组方法是 $[1, 2, 3, 1] = [1, 3], [2, 1]$ ，值为 $B_1 \& B_2 = (1 \oplus 3) \& (2 \oplus 1) = 2 \& 3 = 2$ 。另一种可能的分组方法是 $[1, 2, 3, 1] = [1, 2, 3, 1]$ ，取值较小的 $B_1 = 1 \oplus 2 \oplus 3 \oplus 1 = 1$ 。可以证明不可能达到大于 2 的值。

对于第二个测试用例，最佳分组方法是 $[4, 7, 5, 2, 6, 3] = [7], [5, 3], [6], [4, 2]$ ，值为 $7 \& (5 \oplus 3) \& 6 \& (4 \oplus 2) = 6$ 。



问题H.AGI

时间限制：1 秒内存限制：1024 MB

Menji 博士是人工智能领域的专家。现在，他正在训练Bot，一个AGI（人工游戏智能）。为了教 Bot 玩游戏，他每天都和 Bot 一起玩。今天他们正在玩以下游戏：游戏从 $2n$ 个非负整数、 $a_1, a_2, \dots, a_{2n}$ 和一个数字 S 的序列开始。最初， $S = 0$ 。

Menji 和 Bot 轮流；门吉先行：

- 轮到 Menji，他从序列中选择一个数字 x ，设置 $S \leftarrow S \oplus x$ ，并从序列中删除 x 。请注意， $\oplus$ 是按位 XOR（异或）函数。
- 轮到 Bot，他从序列中选择一个数字 x 并从序列中删除 x 。

当序列中没有数字时，游戏结束。当且仅当最终 $S = 0$ 时，Menji 获胜；否则，Bot 获胜。

Menji 想知道两位选手是否发挥最佳，谁是比赛的胜利者。

输入

输入包含多个测试用例。第一行包含一个整数 T ($1 \leq T \leq 10^5$)，即测试用例的数量。

对于每个测试用例，第一行包含一个整数 n ($1 \leq n \leq 10^5$)，如语句中所述。

第二行包含 $2n$ 整数 $a_1, a_2, \dots, a_{2n}$ ($0 \leq a_i < 2^{30}$)，代表游戏中的整数。保证所有测试用例的 n 之和不超过 2×10^5 。

输出

对于每个测试用例，如果 Menji 能够赢得比赛，则在一行中打印 Menji；否则，在一行中打印 Bot。

例子

standard input	standard output
5	Bot
2	Menji
1 1 3 3	Menji
2	Menji
1 1 1 3	Bot
3	
1 1 4 5 1 4	
3	
1 9 1 9 8 10	
6	
1 1 4 5 1 4 1 9 1 9 8 10	

笔记

对于第一个测试用例，无论 Menji 选择什么数字，Bot 总是可以选择相同的数字，所以 Menji 总是选择 1 和 3， $S = 1 \oplus 3 = 2 \neq 0$ ，所以 Bot 总是可以获胜。

对于第二个测试用例，Menji 在第一回合可以选择 1；无论 Bot 选择什么，Menji 都可以选择另一个 1，所以 Menji 总是收到两个 1， $S = 1 \oplus 1 = 0$ ，所以 Menji 总是可以获胜。



问题一、圆螺丝

时间限制：1 秒内存限制：1024 MB

NIT 喜欢圆形螺丝。他还喜欢 $\oplus$ 运算符，因为它让他想起圆形螺丝，当 $\oplus$ 表示按位异或运算。

关于

将序列 $a_1, a_2, \dots, a_n$ 的值 V_a 定义为 $V_a = a_1 + a_n + \sum_{i=1}^{n-1} (a_i \oplus a_{i+1})$ 。

给定一个序列 $a_1, a_2, \dots, a_n$ ，您可以执行任意次数的以下操作：

- 选择索引 i ($1 \leq i \leq n$)，将 a_i 更改为任意非负整数；这个操作有一个成本 C 。

最小化序列价值与操作所产生的成本之和。换句话说，令 p 为您执行的操作次数， $V_{a'}$ 为操作后的 a 的值，那么您需要最小化 $pC + V_{a'}$ 。

输入

输入包含多个测试用例。输入的第一行包含一个整数 T ($1 \leq T \leq 100$)，测试用例的数量。

对于每个测试用例，第一行包含两个整数 n, C ($2 \leq n \leq 10^5, 0 \leq C < 2^{19}$)，即 n 的长度和顺序和执行操作的成本。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{18}$)，表示序列中的元素。保证所有测试用例的 n 之和不超过 2×10^5 。

输出

对于每个测试用例，在一行中打印一个整数，即 $pC + V_{a'}$ 的最小可能值。

例子

standard input	standard output
3	14
4 4	24
1 4 5 6	29
8 6	
6 6 6 1 1 6 6 6	
6 7	
1 7 2 6 3 5	

笔记

对于第一个测试用例，实现最小值的一种方法是将序列更改为 **1, 1, 1, 0**；粗体数字已更改。最终序列值为 2，进行了 3 次操作；总价值 + 成本为 $2 + 3 \times 4 = 14$ 。

对于第二个测试用例，实现最小值的一种方法是将序列更改为 **6, 6, 6, 6, 6, 6, 6, 6**；最终值为 $12 + 2 \times 6 = 24$ 。



问题 J. 另一个邮箱问题

时间限制: 1.5 秒内存限制: 512 M
B

亚娜、米诺、怀特和哈兹是最好的朋友。

有一天, 教练要求怀特、米诺和哈兹准备一场模拟比赛。Huzz 设计了一个精美的搜索问题, 其复杂度相对于 k 呈指数级增长。他仔细检查了整个问题陈述, 除了一个细节: 他不小心写成了 $k \leq 5 \times 10^5$, 而不是 $k \leq 10$ 。后来, 这个问题出现在一次模拟比赛中。愤怒的参赛者亚娜 (Yana) 来到哈兹 (Huzz) 询问问题该如何解决。但他似乎忘记了什么, 而且他提供的问题陈述看起来也略有不同——

给您一个具有 n 个顶点和 m 个边的有向图, 其中每条边 e 都分配有一个介于 1 和 8 之间的整数权重 $w(e)$ 。

路径是一系列边 $(e_1, e_2, \dots, e_\ell)$, 使得 e_i 的端点是所有 $1 \leq i < \ell$ 的 e_{i+1} 的起点。路径的长度是 ℓ , 即它包含的边数。请注意, 一条路径可能多次包含相同的边。

路径的权重序列是边权重序列 $[w(e_1), w(e_2), \dots, w(e_\ell)]$ 。路径按其权重序列的字典顺序进行比较。

只要两条路径使用不同的边, 即使它们共享相同的顶点序列和权重序列, 它们也被认为是不同的。例如, 如果路径 (e_1, e_2) 和 (e_3, e_4) 都具有权重序列 $[1, 2]$, 并且都遍历顶点 $1 \rightarrow 2 \rightarrow 3$, 则只要 $e_1 \neq e_3$ 或 $e_2 \neq e_4$, 它们仍然是不同的。

怀特想要找到字典顺序最小的 k 路径。由于总输出可能太大了, 哟
只需要输出每条路径的长度。

你

输入

输入的第一行包含三个整数 n, m, k ($2 \leq n \leq 5 \times 10^5, 1 \leq m \leq 5 \times 10^5, 1 \leq k \leq 5 \times 10^5$), 分别代表顶点数、边数和所需的路径数。

接下来的 m 行包含三个整数 x, y, z ($1 \leq x, y \leq n, 1 \leq z \leq 8, x \neq y$), 表示权重为 $w(e) = z$ 的有向边 $e = (x, y)$ 。给定的边集可以包含多个边。

输出

打印 k 行。第 i 行应包含一个整数, 即权重按字典顺序最小的第 i 行的路径长度。如果路径少于 i , 则输出 -1 。



例子

standard input	standard output
5 5 8 2 1 1 3 1 2 4 1 1 1 5 2 5 2 1	1 1 1 2 3 4 5 6
3 4 10 1 2 1 1 2 1 2 3 2 2 3 3	1 1 2 2 2 2 1 1 -1 -1
6 5 15 1 2 3 2 3 5 3 4 2 3 5 1 5 6 4	1 2 1 1 2 3 4 3 1 1 2 3 2 -1 -1

笔记

为简单起见，让 e_j 表示第 j 个输入边。

对于 e 第一个测试用例，字典顺序最小的 8 路径 是：

- 路径(e_1), 权重序列[1]。
- 路径(e_3), 权重序列[1]。
- 路径(e_5), 权重序列[1]。
- 路径(e_5, e_1), 权重序列[1,1]。
- 路径(e_5, e_1, e_4), 权重序列[1,1,2]。
- 路径(e_5, e_1, e_4, e_5), 权重序列[1,1,2,1]。
- 路径(e_5, e_1, e_4, e_5, e_1), 权重序列[1,1,2,1,1]。



- PAtH $(e_5, e_1, e_4, e_5, e_1, e_4)$, weight sequence $[1, 1, 2, 1, 1, 2]$.



问题K.不再后悔

时间限制：4秒
内存限制：1024兆字节

省队选后，怀特陷入了长时间的失望之中。她对自己的结果看不到任何希望。即便如此，怀特还是继续了 NOI 之前的最后训练。除了常规训练之外，她也开始探索在OI时期从未有机会学习的话题——希望在告别之前，不再有遗憾。

甩掉胡思乱想，白突然把注意力集中在眼前的一个问题上，一个平淡无味的数据结构问题——

白色有一个由 n 整数 $a_1, a_2, \dots, a_n$ 组成的序列。她将对这个序列执行 q 操作。每个操作都是以下三种类型之一：

- 1 l r v — 将 v 添加到区间 $[l, r]$ 中的每个元素。
- 2 l r v — 将区间 $[l, r]$ 中的每个元素分配给 v 。
- 3 l r — 查询值 $\sum_{i=l}^r (\min_{j=l}^i a_j) \times (\max_{j=l}^i a_j)$ 模 2^{64} 。

你的任务是模拟所有操作并输出所有查询的结果。

输入

输入第一行包含两个整数 n, q ($1 \leq n, q \leq 2 \times 10^5$)，代表数字元素和操作的数量的

第二行包含 n 整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$)，表示初始元素。

接下来的每一行 q 都以三种格式之一描述一个操作：

- 1 l r v ($1 \leq l \leq r \leq n, -10^9 \leq v \leq 10^9$)，表示相加运算。
- 2 l r v ($1 \leq l \leq r \leq n, 1 \leq v \leq 10^9$)，代表Assign操作。
- 3 l r ($1 \leq l \leq r \leq n$)，代表Query操作。

整个过程中保证每 $1 \leq i \leq n$ 有 $0 \leq a_i \leq 10^9$ 。

输出

对于每个查询操作，在一行中打印一个整数 - 查询模 2^{64} 的结果。



例子

standard input	standard output
5 8	35
2 3 5 4 1	39
3 1 5	40
3 2 4	34
2 4 5 2	177
3 1 5	120
3 2 4	
1 1 2 5	
3 1 5	
3 2 4	
10 20	40
1 2 3 4 5 6 7 8 9 10	156
1 1 10 1	270
3 1 5	120
3 2 9	1202
3 8 10	270
1 2 5 10	118
3 1 5	831
3 2 9	80
3 8 10	33
1 5 9 -5	61
3 1 5	80
3 2 9	40
3 8 10	156
1 2 5 -10	270
3 1 5	
3 2 9	
3 8 10	
1 5 9 5	
3 1 5	
3 2 9	
3 8 10	

笔记

在第一个测试用例中：

原来的序列是2 3 5 4 1，第三次运算后变成2 3 2 2 1，第六次运算后变成7 8 7 7 6。

对于第一个查询，答案为 $\sum_{i=1}^5 (\min_{j=1}^i a_j) \times (\max_{j=1}^i a_j) = 2 \times 2 + 2 \times 3 + 2 \times 5 + 2 \times 5 + 1 \times 5 = 35$ 。

对于第二个查询，答案为 $\sum_{i=2}^4 (\min_{j=2}^i a_j) \times (\max_{j=2}^i a_j) = 3 \times 3 + 3 \times 5 + 3 \times 5 = 39$ 。



问题 L. 另一个排列问题

时间限制: 1.5 秒内存限制: 1024 M
B

亚娜、米诺、怀特和哈兹是最好的朋友。

终于完成了教练布置的艰巨任务，哈兹也得到了休息的机会。他在一个慵懒的下午找到了它，世界似乎在缓慢地爬行，笼罩在即将到来的黄昏温暖的金色薄雾中。一阵微风吹过，只不过是吹动了阳光透过一棵大橡树的叶子倾斜而舞动的尘埃。

在那个昏昏欲睡、舒适的空间里，哈兹在他的软鲨鱼玩具中找到了一种排列。他决定分享它和他的朋友一起玩耍。

米诺喜欢分裂。他可以将这个排列分成几个连续的片段。

亚娜喜欢交换。他可以选择一个连续的段并交换其最大值和最小值元素。m

具体来说，他们可以在一个时间内多次执行以下两种类型的操作：y
命令：

- split: 选择一个长度大于1的连续线段，然后在其中选择一个位置，将其分割成两个相邻的连续线段。例如， $(a_i, \dots, a_j)$ 可以拆分为 $(a_i, \dots, a_k)$ 和 $(a_{k+1}, \dots, a_j)$ ，其中 $i \leq k < j$ 。
- swap: 选择一个连续的段并交换其最大和最小元素。

执行任意数量的操作后，它们将停止执行，并且所有结果段将按其原始顺序合并以形成新的排列。

白喜欢数数。她想知道——他们能获得多少种不同的排列？

由于结果可能非常大，您只需对 998 244 353 求模即可得到答案。

输入

输入的第一行包含一个整数 n ($1 \leq n \leq 500$)，即排列的长度。

第二行包含 n 整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$)，表示排列本身。保证 $\{a_n\}$ 是一个排列。

输出

打印一个整数，即它们可以以 998 244 353 为模获得的不同排列的数量。

例子

standard input	standard output
4 1 4 2 3	10
7 5 1 4 2 6 3 7	340

笔记

在第一个示例中，以下是他们可以获得的所有可能的排列：

(1234)、(1243)、(1423)、(1432)、(4123)、(4132)、(4213)、(4231)、(4312)、(4321)



问题M.又一个01问题

时间限制：5秒
内存限制：1024兆字节

亚娜、米诺、怀特和哈兹是最好的朋友。

米诺最近感到很困惑。由于沉迷于过去在OI方面的失败，他一直在努力规划自己的未来，同时还要管理繁忙的学业日程。他的三个朋友建议他评估任务的重要性并确定优先顺序。

假设有 n 个不同的任务，编号从 1 到 n 。每次，Huzz 选择两个相邻的任务，White 比较它们，Yana 将它们合并为一个任务。Mino 惊讶地发现这个过程意外地形成了一棵线段树，它并不一定在中间分裂。更准确地说，它形成一棵具有 $2n - 1$ 个节点的树，其中每个子树对应一个连续的段。请注意，所有 n 任务都是叶节点，而其他 $n - 1$ 节点各有两个子节点。这些由比较产生的节点定义了其子节点的边值：0 表示较小的节点，1 表示较大的节点。

米诺非常喜欢算计。他将每个任务的权重定义为从任务到根的路径上的边的异或和。他想知道：如果给定权重，有多少种方法来构造这样的树并比较子树？

再一次，米诺不擅长OI，这就是他向你寻求帮助的原因。为了简化问题，只需对 998,244,353 求模即可得到答案。

输入

输入的第一行包含一个正整数 n ($1 \leq n \leq 250000$)，即任务数。

第二行包含长度为 n 的二进制字符串 S ，其中 S_i 表示所有值的按位异或从任务 i 到根的边缘。

es

输出

打印一个整数，答案以 998,244,353 为模。

例子

standard input	standard output
4 0101	6

笔记

有 6 种方法可以构建树并比较子节点：

